

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ
УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ім. Ігоря
СІКОРСЬКОГО»
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ АТОМНОЇ ТА
ТЕПЛОВОЇ ЕНЕРГЕТИКИ

ВІЗУАЛІЗАЦІЯ ГРАФІЧНОЇ ТА ГЕОМЕТРИЧНОЇ ІНФОРМАЦІЇ

РОЗРАХУНКОВА ТА ГРАФІЧНА РОБОТА

Операції над текстурними координатами

Варіант 19

Виконав студент 5-го курсу
групи ТР-51 мп
Лопатецький Михайло

Київ 2025

1 Мета та основні завдання роботи

1.1 Мета

Розширити функціонал попереднього завдання (РАЗ) шляхом додавання операцій масштабування текстури навколо точки заданої користувачем в UV -просторі.

1.2 Завдання

1. Використати наявну систему текстурного відображення з Control task (РАЗ).
2. Реалізувати масштабування текстурних координат навколо точки повороту.
3. Забезпечити можливість переміщення точки вздовж поверхні в UV -просторі за допомогою клавіатури.
4. Клавіші A/D — переміщення по U -параметру, W/S — переміщення по V -параметру.

1.3 Завдання Control task

1. Відрендерити поверхню з трьома застосованими текстурами: дифузною текстурою, яка модулює компонент дифузного освітлення, дзеркальною текстурою, яка модулює компонент дзеркального освітлення, та нормальною текстурою, яка містить дані нормалі для кожного пікселя.
2. Під час ортогоналізації Грама–Шмідта надавати пріоритет нормалі або дотичній, залежно від варіанту.

2 Теоретичні основи

2.1 Текстурні координати та UV-простір

UV-простір є двовимірною системою координат для відображення текстур на тривимірні поверхні. Кожна вершина моделі має координати $(u, v) \in [0, 1] \times [0, 1]$, де u — горизонтальна вісь, v — вертикальна. Текстурні координати інтерполюються між вершинами для кожного фрагмента. Математично текстурне відображення описується як $T(u, v) = \text{texture_color_at}(u, v)$.

У коді це реалізовано через генерацію текстурних координат на основі параметрів поверхні:

```
1 // Model.js
2 // Texture coordinates: u based on r, v based on theta
3 let u = (r - rMin) / (rMax - rMin);
4 let v = theta / (2 * Math.PI);
5 texcoords[i * numTheta + j] = [u, v];
```

2.2 Афінні перетворення текстурних координат

Масштабування текстури навколо точки $P(p_u, p_v)$ виконується за формулою:

$$u' = (u - p_u) \times s + p_u, \quad v' = (v - p_v) \times s + p_v.$$

де:

- (u, v) — оригінальні текстурні координати;
- (u', v') — перетворені координати;
- $P(p_u, p_v)$ — точка масштабування (pivot);
- s — коефіцієнт масштабування

Це перетворення можна представити у вигляді матричних операцій:

Матриця зміщення до початку координат:
$$\begin{bmatrix} 1 & 0 & -p_u \\ 0 & 1 & -p_v \\ 0 & 0 & 1 \end{bmatrix}$$

Матриця масштабування:
$$\begin{bmatrix} s & 0 & 0 \\ 0 & s & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Матриця зворотного зміщення:
$$\begin{bmatrix} 1 & 0 & p_u \\ 0 & 1 & p_v \\ 0 & 0 & 1 \end{bmatrix}$$

Фінальне матричне перетворення:

$$\begin{bmatrix} u' \\ v' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & p_u \\ 0 & 1 & p_v \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} s & 0 & 0 \\ 0 & s & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 1 & 0 & -p_u \\ 0 & 1 & -p_v \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

Це перетворення складається з трьох етапів:

Зміщення: *coord* — *pivot* - перенесення початку координат в точку масштабування

Масштабування: $\times scale$ - застосування коефіцієнта масштабування

Зворотне зміщення: $+pivot$ - повернення до оригінальної системи координат

Такий підхід зберігає положення точки P незмінним під час трансформації. Відповідний фрагмент коду:

```
1 // shader.gpu
2 // Apply texture scaling around pivot point
3 vec2 scaledUV = (tex - uTexturePivot) * uTextureScale + uTexturePivot;
4 vUV = scaledUV;
```

2.3 Обробка граничних умов при трансформаціях

При масштабуванні з коефіцієнтом відмінним від 1 виникає необхідність обробки виходу за межі $[0, 1]$. Функція `fract()` забезпечує модульну арифметику для створення ефекту повторення текстури. Це математично еквівалентно $wrappedUV = vUV - floor(vUV)$, що гарантує значення в діапазоні $[0, 1)$.

3 Програмний етап

Передмова

Була додана функція для анімації обрешітання точки освітлення навколо фігури, якої не вистачало в минулій версії роботи.

```
1 function animate() {  
2     draw();  
3     requestAnimationFrame(animate);  
4 }
```

3.1 Архітектура розширення системи

Система розширює функціонал РАЗ додаванням двох нових uniform-змінних: `uTextureScale` для коефіцієнта масштабування та `uTexturePivot` для точки трансформації.

```
1 // shader.jpu  
2 // Texture transformation uniforms  
3 uniform float uTextureScale;  
4 uniform vec2 uTexturePivot;
```

Глобальні змінні `textureScale` та `texturePivot` зберігають поточний стан трансформацій.

```
1 // main.js  
2 // Texture transformation variables  
3 let textureScale = 1.0;  
4 let texturePivot = [0.5, 0.5]; // Initial pivot point in UV space [u, v]
```

Модифікований вершинний шейдер застосовує трансформації до текстурних координат перед інтерполяцією.

3.2 Механізм обробки введення користувача

Функція `handleKeyPress` обробляє клавіатурні події з кроком 0.05 для переміщення точки та 0.1 для масштабування. Клавіші *W/S* змінюють *v*-координату, *A/D* — *u*-координату, *Q/E* — коефіцієнт масштабування. Функція `Math.min()` та `Math.max()` гарантують значення в межах $[0, 1]$ для координат та $[0.1, 3.0]$ для масштабу. Після кожної зміни викликається `draw()` для оновлення відображення.

```
1 // main.js  
2 // Keyboard event handler for texture transformation  
3 function handleKeyPress(event) {  
4     const step = 0.05;  
5     const scaleStep = 0.1;  
6  
7     switch(event.key.toLowerCase()) {  
8         case 'w':  
9             // Move pivot up (increase v)  
10            texturePivot[1] = Math.min(1.0, texturePivot[1] + step);  
11            break;  
12         case 's':  
13            // Move pivot down (decrease v)  
14            texturePivot[1] = Math.max(0.0, texturePivot[1] - step);
```

```

15         break;
16     case 'a':
17         // Move pivot left (decrease u)
18         texturePivot[0] = Math.max(0.0, texturePivot[0] - step);
19         break;
20     case 'd':
21         // Move pivot right (increase u)
22         texturePivot[0] = Math.min(1.0, texturePivot[0] + step);
23         break;
24     case 'q':
25         // Scale texture up
26         textureScale = Math.min(3.0, textureScale + scaleStep);
27         break;
28     case 'e':
29         // Scale texture down
30         textureScale = Math.max(0.1, textureScale - scaleStep);
31         break;
32     case 'r':
33         // Reset transformations
34         textureScale = 1.0;
35         texturePivot = [0.5, 0.5];
36         break;
37     default:
38         return; // Ignore other keys
39 }
40
41 console.log('Texture Scale: ${textureScale.toFixed(2)}, Pivot: [${texturePivot
42 [0].toFixed(2)}, ${texturePivot[1].toFixed(2)}]');
43 draw(); // Redraw with new transformations

```

3.3 Модифікація шейдерів для підтримки трансформацій

Вершинний шейдер тепер містить масштабування навколо заданої точки. Фрагментний шейдер використовує `fract()` функцію для обробки виходу за межі текстурних координат (Див. підрозділи 2.2, 2.3). TBN-матриця продовжує коректно працювати з нормальми після трансформації координат.

3.4 Інтеграція з існуючим конвеєром рендерингу

У функції `draw()` додано передачу параметрів трансформації:

```

1 // main.js
2 // Set texture transformation uniforms
3 gl.uniform1f(shProgram.iTextureScale, textureScale);
4 gl.uniform2fv(shProgram.iTexturePivot, texturePivot);

```

Об'єкт `ShaderProgram` розширено властивостями `iTextureScale` та `iTexturePivot` для зберігання локацій `uniform`-змінних. Система освітлення залишається незмінною та функціонує незалежно від текстурних трансформацій.

3.5 Управління станом трансформацій

Початковий стан: $textureScale = 1.0, texturePivot = [0.5, 0.5]$. Клавiша R скидає трансформації до початкових значень. Кожна зміна параметрів логується у консоль для налагодження. Інтерфейс користувача оновлює відображення поточних значень кожні 100 мс через `setInterval(updateTextureInfo, 100)`.

```
1 // index.html
2 setInterval(updateTextureInfo, 100);
3
4 function updateTextureInfo() {
5   document.getElementById('scaleValue').textContent = textureScale.toFixed(2);
6   document.getElementById('pivotU').textContent = texturePivot[0].toFixed(2);
7   document.getElementById('pivotV').textContent = texturePivot[1].toFixed(2);
8 }
```

4 Інструкція користувача

4.1 Запуск та ініціалізація програми

Відкрити `index.html` у браузері з підтримкою WebGL. Переконаватися у наявності текстур у папці `Utils/textures/`. Система автоматично ініціалізує поверхню з параметрами за замовчуванням. При відсутності текстур використовуються fallback-версії.

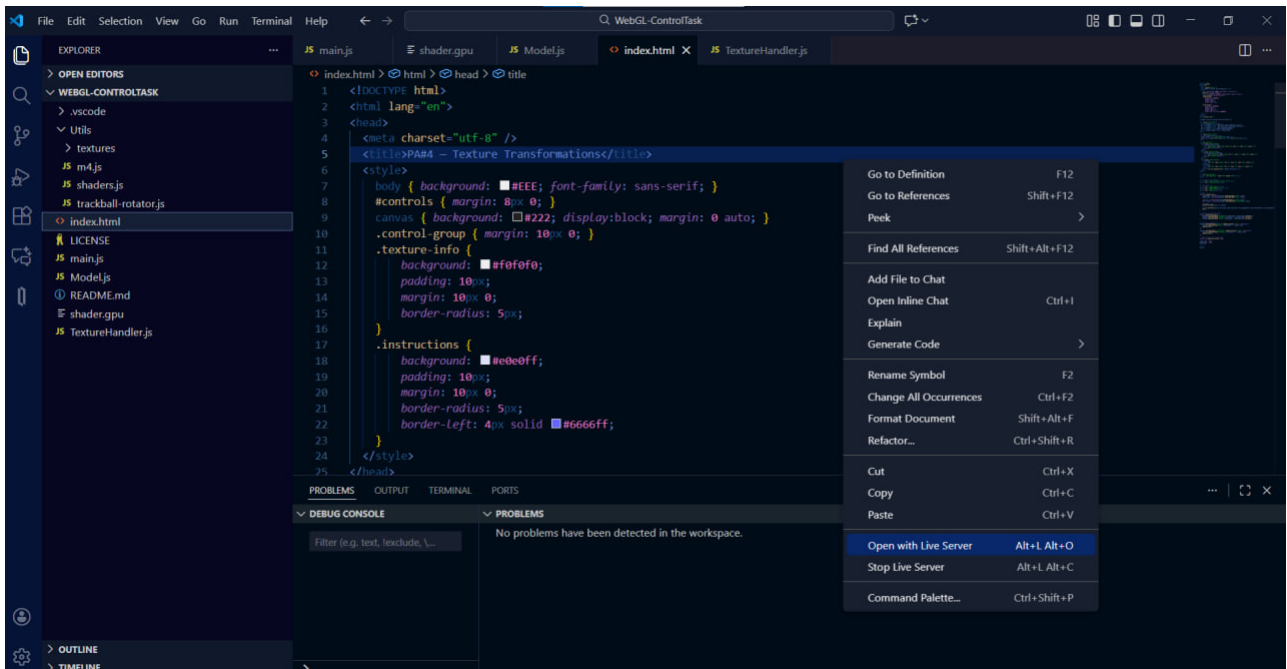


Рис. 1

4.2 Елементи керування трансформаціями

Клавіші W/S переміщують точку масштабування по вертикалі (v -координата). Клавіші A/D переміщують точку по горизонталі (u -координата). Клавіші Q/E збільшують/зменшують масштаб текстури. Клавіша R скидає всі трансформації. Миша обертає камеру навколо поверхні.

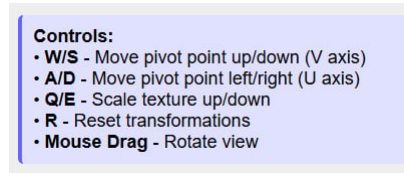


Рис. 2

4.3 Візуалізація параметрів трансформації

Панель Texture Info відображає поточний масштаб з точністю до двох знаків. Координати pivot точки показуються у форматі $[u, v]$. Слайдери керують деталізацією поверхні: $GranularityU$ контролює параметр dr , $GranularityV$ – $dTheta$. Кнопка Update Surface переобчислює геометрію.

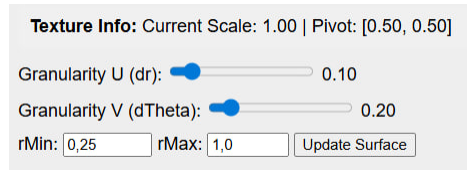


Рис. 3

4.4 Практичні сценарії використання

Для масштабування від центру встановіть $pivot[0.5, 0.5]$, використовуйте Q/E . Для асиметричного масштабування перемістіть $pivot$ до $[0.0, 0.5]$ клавішами A , потім масштабуйте. Спостерігайте різну поведінку при русі по U та V через нелінійність UV -координат на криволінійній поверхні.

Зауваження: При масштабі $\neq 1$ виникають візуальні спотворення через модульне повторення текстури.

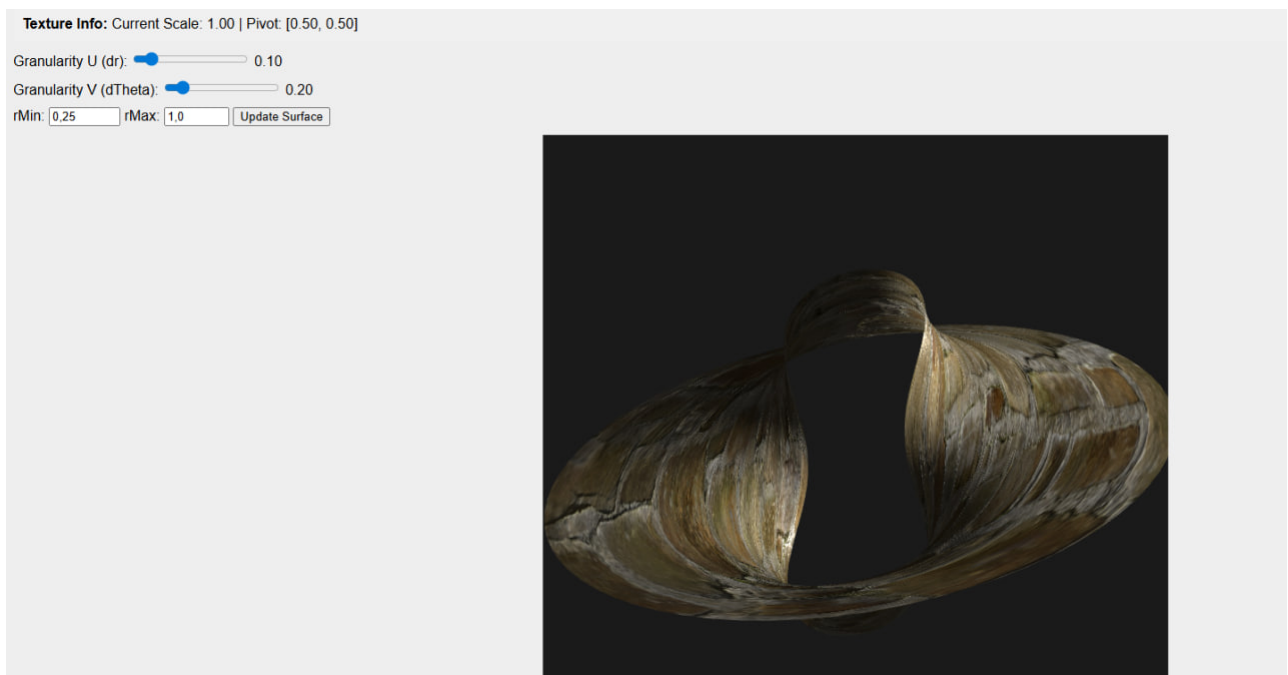


Рис. 4

5 Висновок

У ході роботи було розширено функціонал попередньої системи текстурного відображення шляхом додавання механізму масштабування текстури навколо довільної точки UV -простору. Реалізовано повноцінну підтримку афінних перетворень текстурних координат, інтегровано нові `uniform`-змінні та оновлено шейдери для коректної обробки трансформацій. Додано зручне керування точкою масштабування та коефіцієнтом масштабу за допомогою клавіатури, а також систему візуального моніторингу параметрів. Окремо було виправлено анімацію руху джерела освітлення.

Додатки

Файли проекту в яких були внесені зміни:

main.js:

```
1 'use strict';
2
3 let gl;
4 let surface;
5 let shProgram;
6 let spaceball;
7
8 // Texture transformation variables
9 let textureScale = 1.0;
10 let texturePivot = [0.5, 0.5]; // Initial pivot point in UV space [u, v]
11
12 function ShaderProgram(name, program) {
13     this.name = name;
```

```

14     this.prog = program;
15
16     this.iAttribVertex = gl.getAttribLocation(program, "vertex");
17     this.iAttribNormal = gl.getAttribLocation(program, "normal");
18     this.iAttribTangent = gl.getAttribLocation(program, "tangent");
19     this.iAttribTexCoords = gl.getAttribLocation(program, "tex");
20
21     this.iModelViewProjectionMatrix = gl.getUniformLocation(program, "
    ModelViewProjectionMatrix");
22     this.iModelViewMatrix = gl.getUniformLocation(program, "ModelViewMatrix");
23     this.iNormalMatrix = gl.getUniformLocation(program, "NormalMatrix");
24
25     this.iDiffuseTex = gl.getUniformLocation(program, "uDiffuseTex");
26     this.iSpecularTex = gl.getUniformLocation(program, "uSpecularTex");
27     this.iNormalMap = gl.getUniformLocation(program, "uNormalMap");
28
29     this.iLightPosEye = gl.getUniformLocation(program, "uLightPosEye");
30     this.iAmbientColor = gl.getUniformLocation(program, "uAmbientColor");
31     this.iLightColor = gl.getUniformLocation(program, "uLightColor");
32     this.iShininess = gl.getUniformLocation(program, "uShininess");
33
34     // New uniforms for texture transformation
35     this.iTextureScale = gl.getUniformLocation(program, "uTextureScale");
36     this.iTexturePivot = gl.getUniformLocation(program, "uTexturePivot");
37
38     this.Use = function() {
39         gl.useProgram(this.prog);
40     };
41 }
42
43 // Function to compute inverse of 3x3 matrix from 4x4 matrix
44 function toInverseMat3(mat4) {
45     // Extract upper-left 3x3 part from 4x4 matrix
46     const m00 = mat4[0], m01 = mat4[1], m02 = mat4[2];
47     const m10 = mat4[4], m11 = mat4[5], m12 = mat4[6];
48     const m20 = mat4[8], m21 = mat4[9], m22 = mat4[10];
49
50     // Compute determinant
51     const det = m00 * (m11 * m22 - m12 * m21) -
52                 m01 * (m10 * m22 - m12 * m20) +
53                 m02 * (m10 * m21 - m11 * m20);
54
55     // Return identity matrix if determinant is too small
56     if (Math.abs(det) < 1e-8) {
57         return [1, 0, 0, 0, 1, 0, 0, 0, 0, 1];
58     }
59
60     const invDet = 1.0 / det;
61
62     // Compute inverse matrix
63     return [
64         (m11 * m22 - m12 * m21) * invDet,
65         (m02 * m21 - m01 * m22) * invDet,
66         (m01 * m12 - m02 * m11) * invDet,
67         (m12 * m20 - m10 * m22) * invDet,
68         (m00 * m22 - m02 * m20) * invDet,

```

```

69         (m02 * m10 - m00 * m12) * invDet,
70         (m10 * m21 - m11 * m20) * invDet,
71         (m01 * m20 - m00 * m21) * invDet,
72         (m00 * m11 - m01 * m10) * invDet
73     ];
74 }
75
76 function draw() {
77     // Clear canvas
78     gl.clearColor(0.1, 0.1, 0.1, 1);
79     gl.clear(gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT);
80
81     // Setup projection and view matrices
82     const projection = m4.perspective(Math.PI / 8, 1, 8, 12);
83     const modelView = spaceball.getViewMatrix();
84     const translateToPointZero = m4.translation(0, 0, -10);
85     const modelViewMatrix = m4.multiply(translateToPointZero, modelView);
86     const modelViewProjection = m4.multiply(projection, modelViewMatrix);
87
88     // Set matrix uniforms
89     gl.uniformMatrix4fv(shProgram.iModelViewMatrix, false, modelViewMatrix);
90     gl.uniformMatrix4fv(shProgram.iModelViewProjectionMatrix, false,
91         modelViewProjection);
92
93     // Compute and set normal matrix
94     const normalMatrix = toInverseMat3(modelViewMatrix);
95     gl.uniformMatrix3fv(shProgram.iNormalMatrix, false, normalMatrix);
96
97     // Animated light orbiting around the surface
98     const time = performance.now() / 1000;
99     const lightPosEye = [5 * Math.cos(time), 3.0, 5 * Math.sin(time)];
100     gl.uniform3fv(shProgram.iLightPosEye, lightPosEye);
101
102     // Lighting parameters
103     gl.uniform3fv(shProgram.iAmbientColor, [0.2, 0.2, 0.2]);
104     gl.uniform3fv(shProgram.iLightColor, [1.0, 1.0, 1.0]);
105     gl.uniform1f(shProgram.iShininess, 64.0);
106
107     // Set texture transformation uniforms
108     gl.uniform1f(shProgram.iTextureScale, textureScale);
109     gl.uniform2fv(shProgram.iTexturePivot, texturePivot);
110
111     // Set texture units
112     gl.uniform1i(shProgram.iDiffuseTex, 0);
113     gl.uniform1i(shProgram.iSpecularTex, 1);
114     gl.uniform1i(shProgram.iNormalMap, 2);
115
116     // Draw the surface
117     surface.Draw();
118 }
119
120 function animate() {
121     draw();
122     requestAnimationFrame(animate);
123 }

```

```

124 // Keyboard event handler for texture transformation
125 function handleKeyPress(event) {
126     const step = 0.05;
127     const scaleStep = 0.1;
128
129     switch(event.key.toLowerCase()) {
130         case 'w':
131             // Move pivot up (increase v)
132             texturePivot[1] = Math.min(1.0, texturePivot[1] + step);
133             break;
134         case 's':
135             // Move pivot down (decrease v)
136             texturePivot[1] = Math.max(0.0, texturePivot[1] - step);
137             break;
138         case 'a':
139             // Move pivot left (decrease u)
140             texturePivot[0] = Math.max(0.0, texturePivot[0] - step);
141             break;
142         case 'd':
143             // Move pivot right (increase u)
144             texturePivot[0] = Math.min(1.0, texturePivot[0] + step);
145             break;
146         case 'q':
147             // Scale texture up
148             textureScale = Math.min(3.0, textureScale + scaleStep);
149             break;
150         case 'e':
151             // Scale texture down
152             textureScale = Math.max(0.1, textureScale - scaleStep);
153             break;
154         case 'r':
155             // Reset transformations
156             textureScale = 1.0;
157             texturePivot = [0.5, 0.5];
158             break;
159         default:
160             return; // Ignore other keys
161     }
162
163     console.log('Texture Scale: ${textureScale.toFixed(2)}, Pivot: [${texturePivot
164     [0].toFixed(2)}, ${texturePivot[1].toFixed(2)}]');
165     draw(); // Redraw with new transformations
166 }
167
168 function initGL() {
169     // Create and use shader program
170     const prog = createProgram(gl, vertexShaderSource, fragmentShaderSource);
171     shProgram = new ShaderProgram('PhongTex', prog);
172     shProgram.Use();
173
174     // Generate surface data
175     const data = {};
176     CreateSurfaceData(data);
177
178     // Create and buffer surface model
179     surface = new Model('Surface');

```

```

179     surface.BufferData(data.verticesF32, data.normalsF32, data.tangentsF32, data.
      texcoordsF32, data.indicesU16);
180
181     // Load textures (fallback textures will be used if files not found)
182     surface.idTextureDiffuse = LoadTexture('Utils/textures/diffuse.jpg');
183     surface.idTextureSpecular = LoadTexture('Utils/textures/specular.jpg');
184     surface.idTextureNormal = LoadTexture('Utils/textures/normal.jpg');
185
186     // Enable depth testing
187     gl.enable(gl.DEPTH_TEST);
188 }
189
190 function init() {
191     const canvas = document.getElementById("webglcanvas");
192     gl = canvas.getContext("webgl");
193     if (!gl) {
194         alert("WebGL not supported!");
195         return;
196     }
197
198     try {
199         initGL();
200         spaceball = new TrackballRotator(canvas, draw, 0);
201
202         // Add keyboard event listener
203         window.addEventListener('keydown', handleKeyPress);
204
205         animate();
206     } catch (error) {
207         console.error("Error during initialization:", error);
208         alert("Error initializing WebGL application: " + error.message);
209     }
210 }
211
212 function createProgram(gl, vShader, fShader) {
213     const vsh = gl.createShader(gl.VERTEX_SHADER);
214     gl.shaderSource(vsh, vShader);
215     gl.compileShader(vsh);
216     if (!gl.getShaderParameter(vsh, gl.COMPILE_STATUS)) {
217         const error = gl.getShaderInfoLog(vsh);
218         gl.deleteShader(vsh);
219         throw new Error('Vertex shader compilation error: ' + error);
220     }
221
222     const fsh = gl.createShader(gl.FRAGMENT_SHADER);
223     gl.shaderSource(fsh, fShader);
224     gl.compileShader(fsh);
225     if (!gl.getShaderParameter(fsh, gl.COMPILE_STATUS)) {
226         const error = gl.getShaderInfoLog(fsh);
227         gl.deleteShader(fsh);
228         gl.deleteShader(vsh);
229         throw new Error('Fragment shader compilation error: ' + error);
230     }
231
232     const prog = gl.createProgram();
233     gl.attachShader(prog, vsh);

```

```

234 gl.attachShader(prog, fsh);
235 gl.linkProgram(prog);
236 if (!gl.getProgramParameter(prog, gl.LINK_STATUS)) {
237     const error = gl.getProgramInfoLog(prog);
238     gl.deleteProgram(prog);
239     gl.deleteShader(vsh);
240     gl.deleteShader(fsh);
241     throw new Error('Program linking error: ' + error);
242 }
243
244 // Clean up shaders after linking
245 gl.deleteShader(vsh);
246 gl.deleteShader(fsh);
247
248 return prog;
249 }

```

shader.jpu:

```

1 const vertexShaderSource = `
2 attribute vec3 vertex;
3 attribute vec3 normal;
4 attribute vec3 tangent;
5 attribute vec2 tex;
6
7 uniform mat4 ModelViewMatrix;
8 uniform mat4 ModelViewProjectionMatrix;
9 uniform mat3 NormalMatrix;
10
11 // Texture transformation uniforms
12 uniform float uTextureScale;
13 uniform vec2 uTexturePivot;
14
15 varying vec3 vT_eye;
16 varying vec3 vN_eye;
17 varying vec3 vB_eye;
18 varying vec2 vUV;
19 varying vec3 vPosEye;
20
21 void main() {
22     // position in eye space
23     vec4 posEye4 = ModelViewMatrix * vec4(vertex, 1.0);
24     vPosEye = posEye4.xyz;
25
26     // transform normal & tangent to eye space (as vectors)
27     vec3 N_eye = normalize( NormalMatrix * normal );
28     vec3 T_eye = normalize( NormalMatrix * tangent );
29
30     // Gram-Schmidt orthogonalization with PRIORITY = TANGENT:
31     // keep T as primary, orthogonalize N w.r.t T
32     T_eye = normalize(T_eye);
33     N_eye = normalize( N_eye - dot(N_eye, T_eye) * T_eye );
34
35     // compute bitangent (B) as cross(T, N)
36     vec3 B_eye = normalize( cross(T_eye, N_eye) );
37
38     vT_eye = T_eye;

```

```

39     vN_eye = N_eye;
40     vB_eye = B_eye;
41
42     // Apply texture scaling around pivot point
43     vec2 scaledUV = (tex - uTexturePivot) * uTextureScale + uTexturePivot;
44     vUV = scaledUV;
45
46     gl_Position = ModelViewProjectionMatrix * vec4(vertex, 1.0);
47 }
48 ';
49
50 const fragmentShaderSource = '
51 #ifdef GL_FRAGMENT_PRECISION_HIGH
52     precision highp float;
53 #else
54     precision mediump float;
55 #endif
56
57 varying vec3 vT_eye;
58 varying vec3 vN_eye;
59 varying vec3 vB_eye;
60 varying vec2 vUV;
61 varying vec3 vPosEye;
62
63 uniform sampler2D uDiffuseTex;
64 uniform sampler2D uSpecularTex;
65 uniform sampler2D uNormalMap;
66
67 uniform vec3 uLightPosEye;
68 uniform vec3 uAmbientColor;
69 uniform vec3 uLightColor;
70 uniform float uShininess;
71
72 void main() {
73     // Apply texture wrapping for scaled coordinates
74     vec2 wrappedUV = fract(vUV);
75
76     // read textures with wrapped coordinates
77     vec3 diffCol = texture2D(uDiffuseTex, wrappedUV).rgb;
78     vec3 specCol = texture2D(uSpecularTex, wrappedUV).rgb;
79
80     // normal map in tangent space (sampled in [0,1])
81     vec3 nmap = texture2D(uNormalMap, wrappedUV).rgb;
82     vec3 n_ts = normalize(nmap * 2.0 - 1.0); // map to [-1,1]
83
84     // reconstruct normal in eye space using TBN matrix
85     mat3 TBN = mat3(
86         normalize(vT_eye),
87         normalize(vB_eye),
88         normalize(vN_eye)
89     );
90     vec3 N_eye = normalize( TBN * n_ts );
91
92     // lighting vectors
93     vec3 L = normalize(uLightPosEye - vPosEye);
94     vec3 V = normalize(-vPosEye); // viewer direction in eye space

```

```

95     vec3 H = normalize(L + V); // half vector for Blinn-Phong
96
97     float lambert = max(dot(N_eye, L), 0.0);
98     float spec = 0.0;
99     if (lambert > 0.0) {
100         spec = pow(max(dot(N_eye, H), 0.0), uShininess);
101     }
102
103     vec3 ambient = uAmbientColor * diffCol;
104     vec3 diffuse = uLightColor * diffCol * lambert;
105     vec3 specular = uLightColor * specCol * spec;
106
107     vec3 color = ambient + diffuse + specular;
108     gl_FragColor = vec4(color, 1.0);
109 }
110 ';

```

index.html:

```

1  const vertexShaderSource = '
2  attribute vec3 vertex;
3  attribute vec3 normal;
4  attribute vec3 tangent;
5  attribute vec2 tex;
6
7  uniform mat4 ModelViewMatrix;
8  uniform mat4 ModelViewProjectionMatrix;
9  uniform mat3 NormalMatrix;
10
11 // Texture transformation uniforms
12 uniform float uTextureScale;
13 uniform vec2 uTexturePivot;
14
15 varying vec3 vT_eye;
16 varying vec3 vN_eye;
17 varying vec3 vB_eye;
18 varying vec2 vUV;
19 varying vec3 vPosEye;
20
21 void main() {
22     // position in eye space
23     vec4 posEye4 = ModelViewMatrix * vec4(vertex, 1.0);
24     vPosEye = posEye4.xyz;
25
26     // transform normal & tangent to eye space (as vectors)
27     vec3 N_eye = normalize( NormalMatrix * normal );
28     vec3 T_eye = normalize( NormalMatrix * tangent );
29
30     // Gram-Schmidt orthogonalization with PRIORITY = TANGENT:
31     // keep T as primary, orthogonalize N w.r.t T
32     T_eye = normalize(T_eye);
33     N_eye = normalize( N_eye - dot(N_eye, T_eye) * T_eye );
34
35     // compute bitangent (B) as cross(T, N)
36     vec3 B_eye = normalize( cross(T_eye, N_eye) );
37
38     vT_eye = T_eye;

```

```

39     vN_eye = N_eye;
40     vB_eye = B_eye;
41
42     // Apply texture scaling around pivot point
43     vec2 scaledUV = (tex - uTexturePivot) * uTextureScale + uTexturePivot;
44     vUV = scaledUV;
45
46     gl_Position = ModelViewProjectionMatrix * vec4(vertex, 1.0);
47 }
48 ';
49
50 const fragmentShaderSource = '
51 #ifdef GL_FRAGMENT_PRECISION_HIGH
52     precision highp float;
53 #else
54     precision mediump float;
55 #endif
56
57 varying vec3 vT_eye;
58 varying vec3 vN_eye;
59 varying vec3 vB_eye;
60 varying vec2 vUV;
61 varying vec3 vPosEye;
62
63 uniform sampler2D uDiffuseTex;
64 uniform sampler2D uSpecularTex;
65 uniform sampler2D uNormalMap;
66
67 uniform vec3 uLightPosEye;
68 uniform vec3 uAmbientColor;
69 uniform vec3 uLightColor;
70 uniform float uShininess;
71
72 void main() {
73     // Apply texture wrapping for scaled coordinates
74     vec2 wrappedUV = fract(vUV);
75
76     // read textures with wrapped coordinates
77     vec3 diffCol = texture2D(uDiffuseTex, wrappedUV).rgb;
78     vec3 specCol = texture2D(uSpecularTex, wrappedUV).rgb;
79
80     // normal map in tangent space (sampled in [0,1])
81     vec3 nmap = texture2D(uNormalMap, wrappedUV).rgb;
82     vec3 n_ts = normalize(nmap * 2.0 - 1.0); // map to [-1,1]
83
84     // reconstruct normal in eye space using TBN matrix
85     mat3 TBN = mat3(
86         normalize(vT_eye),
87         normalize(vB_eye),
88         normalize(vN_eye)
89     );
90     vec3 N_eye = normalize( TBN * n_ts );
91
92     // lighting vectors
93     vec3 L = normalize(uLightPosEye - vPosEye);
94     vec3 V = normalize(-vPosEye); // viewer direction in eye space

```

```

95     vec3 H = normalize(L + V); // half vector for Blinn-Phong
96
97     float lambert = max(dot(N_eye, L), 0.0);
98     float spec = 0.0;
99     if (lambert > 0.0) {
100         spec = pow(max(dot(N_eye, H), 0.0), uShininess);
101     }
102
103     vec3 ambient = uAmbientColor * diffCol;
104     vec3 diffuse = uLightColor * diffCol * lambert;
105     vec3 specular = uLightColor * specCol * spec;
106
107     vec3 color = ambient + diffuse + specular;
108     gl_FragColor = vec4(color, 1.0);
109 }
110 ‘;

```